A Private Blockchain for Jointly Signed Negotiated Rates Between Providers and Payers

Abhinav Srivastava[1]

[1]**Independent Researcher; abhinavmirdev@gmail.com**

Abstract

US health care providers and health plans must each publish the prices they negotiated with each other. The files describe the same contracts from two sides, and nothing checks one against the other. A 2026 study of 450 hospitals and two plans found that about one in ten paired prices were the same. We present CORAL, a shared ledger on which a provider and a payer both sign each negotiated price with the plan, billing code, setting, and dates that identify it. A published price is then checked against the signed record, and a wrong price is traced to the file that published it. We signed one provider's full price list (262,535 records, 20 payers) and checked 363,690 published prices against the ledger. We paired five providers' files with one Aetna file on 16 billing codes and measured their agreement. A synthetic run traces each remaining difference to one side.

Introduction

Since 2021, each hospital in the United States must publish a machine-readable file of its standard charges, including the rate it negotiated with each payer and plan, under the Hospital Price Transparency rule (45 CFR Part 180) [1]. Since 2022, each group health plan and issuer must publish in-network negotiated rates for every provider under the Transparency in Coverage rule [2]. The two programs publish the same contract terms from two sides. Each file is produced by one party under its own rule, and no clause of either rule asks the other party to review or confirm it [1, 2]. The only attestation is the hospital's affirmation of its own file, added in CY2024 [11].

The two sides do not agree. Trilliant Health compared the files of 450 hospitals with the files of Aetna and UnitedHealthcare on 16 common codes and found that 9.0% of inpatient pairs, 10.8% of gastroenterology pairs, and 11.7% of orthopedic pairs matched exactly; about half fell within 10%; and 17.9% of inpatient and gastroenterology pairs differed by more than 50% [3]. The comparison cannot say which file is correct, because neither file links a line to a contract version, an effective date, or the other party.

A negotiated rate is a contract term that two parties agreed. We take the position that the published record of that term must carry both parties' signatures. CORAL (Committed Ratesheet on Ledger) is a registry on which a provider and a payer jointly sign each negotiated rate before either publishes it. The record carries the fields that identify one rate and the fields that give its value, dates, and contract version. The published files cite the record. A verifier with a file and read access to the registry checks each line with one lookup, and a line that departs from the signed record is attributed to the file that departs.

This paper makes four contributions. First, a record format for a negotiated rate that both public file formats can be mapped to, with a key that we test against a full real rate sheet. Second, a registry contract with a key hierarchy under which validity is judged at commit time, revocations cannot be backdated, root replacement is delayed and can be vetoed, and one record is live per key at any time. Third, a full-sheet run on one provider's public file: all 262,535 rates signed and committed, every published line verified, with throughput, bytes, and storage measured. Fourth, a comparison of five providers' files with one Aetna file on the 16 Trilliant codes, with the data discovery rate reported as a finding, and a synthetic attribution run.

Background

The two files

The provider file, in the CMS tall CSV format, has one row per item and payer plan. The columns that identify a rate are the billing code and its type (CPT, HCPCS, MS-DRG), modifiers, setting (inpatient, outpatient, or both), billing class (facility or professional), drug unit of measure and type, payer name, and plan name. The columns that give its value are the gross charge, the negotiated dollar amount, the negotiated percentage, the negotiated algorithm, and the methodology (fee schedule, case rate, per diem, percent of total billed charges, or other) [4]. The payer file, in the Transparency in Coverage JSON schema, has one item per billing code, with negotiated rates for groups of providers identified by NPI and tax identification number, each with a negotiated type, a billing class, a list of place-of-service codes, and modifiers [5].

What a negotiated rate is

Two terms matter. The chargemaster is the provider's list price for every item. The provider sets it alone; it is not negotiated. The CMS file calls it the gross charge. The negotiated rate is the price the provider and the payer agreed for one code under one plan. It is contractual. The CMS file calls it the payer-specific negotiated charge. A negotiated rate takes one of three forms: a fixed amount, a percent of the chargemaster price, or a percent of a reference fee schedule. In the provider file we profile, 36% of rows are percent-of-charges rates. Such a rate changes value when the chargemaster changes, and the payer never signs the chargemaster.

Why the files disagree

Four causes are visible in the files themselves. First, the identifying fields do not line up: the provider file has setting, the payer file has place-of-service codes; the provider file has a drug unit, the payer file has none; plan names are free text on the provider side and plan clusters on the payer side. Second, each file is a snapshot with one date, and contracts change mid-year. Third, the provider file holds many items under one billing code, with different list prices and sometimes different fixed amounts. Fourth, a percent-of-charges line gives a dollar result that depends on the chargemaster version in force when the file was built. None of these causes can be resolved by comparing the two files, because neither file says what the contract term was.

Related work

Price transparency studies. Jiang and colleagues measured compliance with the hospital rule across the national landscape [6]. Wang and colleagues found Transparency in Coverage prices consistent with other commercial sources [7], and Henderson and Mouslim cross-validated insurer and hospital files [8]. Turquoise Health [9] and PatientRightsAdvocate.org [10] publish recurring reports on file availability and quality. Trilliant Health's 2026 study is the direct motivation for this paper [3]. All of these compare one-party files. None proposes a record that both parties sign.

One-party attestation. The CY2024 OPPS final rule added an affirmation statement to the hospital file: the hospital attests that the file is true, accurate, and complete, and must disclose an estimated allowed amount when a dollar rate cannot be given [11]. This is the closest regulatory precedent. It binds one party to its own file. CORAL binds both parties to one record.

Ledgers in health care. Kuo, Kim, and Ohno-Machado survey distributed ledger applications in biomedicine and give the conditions under which a ledger adds value over a database: several parties that do not trust one operator, a shared need for an append-only history, and no acceptable neutral host [12]. Claims and directory networks on permissioned ledgers exist: Change Healthcare's claims network on Hyperledger Fabric [13], the Synaptic Health Alliance provider directory [14], and ProCredEx for practitioner credentials [15]. None holds a contract term with both parties' signatures.

Transparency logs. Certificate Transparency [16], Trillian, and the IETF SCITT architecture [17] give an append-only log under one operator with inclusion proofs. PeerReview [18] and SUNDR [19] detect equivocation by a log operator. Zieglmeier shows that an append to a chain proves the sender's action and not the counterparty's receipt [20]; CORAL answers this by requiring the counterparty's signature inside the record. A one-operator log would serve CORAL if providers, payers, and the regulator accepted one of them as the operator. We return to this question in the Discussion.

Signing. EIP-712 defines typed structured data signing so that a signing tool shows the fields to the person who signs and binds the signature to one deployment [21]. We use it for every record.

Methods

The record

A rate record has a key and a value. The key is the set of fields that identify one rate: payer id, plan id, provider id (the contracting tax identification number), billing NPI, billing code with its type, modifiers, setting (inpatient or outpatient), billing class (facility or professional), and drug unit of measure. Two records equal on all key fields are the same rate. The value is the formula (fixed, percent of charges, or percent of reference), the amount in cents or basis points, the start and end dates, a contract version label, a reference version (the chargemaster or fee-schedule version

a percent rate applies to), and the id of the record this one supersedes, if any.

The setting and billing class fields come from the CMS file. The payer file carries billing class directly (institutional or professional) and, for professional prices, setting through place-of-service codes: 21 is inpatient; 19, 22, 23, and 24 are outpatient. An institutional price carries no place of service. A DRG prices an admission and is inpatient; any other institutional price is taken as applying in both settings, as the CMS file's "both" does, and the parser counts how often it did so. A professional price with no place of service has no setting, and the parser reports it rather than guessing. The payer file has no drug unit.

Four other record types support the rate record. An Entity binds an entity id to a root public key. A Delegation authorizes a signing key for an entity over a time window. A Revocation ends a signing key from a time at or after the block that carries it. A Chargemaster Version is the provider's signature over a version label, the SHA-256 of the chargemaster file, and an effective date; a percent-of-charges record names it, so the dollar result recomputes from signed inputs.

Identity and keys

The provider's identity root is its tax identification number, because contracts are signed at that level and the payer file carries it. The payer's root is a payer id; the prototype uses an opaque string, and a deployment would use the HIOS issuer id or the X12 payer id. CMS is the registrar. CMS already receives both files and already proofs both kinds of organization through PECOS and the Identity and Access Management System. The registrar binds an entity id to a root key once, after out-of-band proofing, and never signs a rate.

Each entity holds one root key offline and delegates one or more signing keys to the system that produces its rate sheet. The root signs delegations and revocations only; it cannot sign a rate, because the registry checks a rate signer against delegations and a root has none. A verifier answers "was key K authorized for entity E at time T" from the registry alone.

Validity is judged at the block timestamp of the commit, which consensus sets, not the signer. A revocation must take effect at or after the block that carries it, so a root cannot backdate a revocation to before a record its signing key committed; records committed before a revocation stand. Root replacement after loss is proposed by the registrar, takes effect after seven days, and the current root can veto it during that window with its signature. Without the delay the registrar would be a single operator in all but name.

The registry

The registry is one contract on a permissioned Ethereum-compatible network. Each action carries EIP-712 signatures over a typed structure; the domain binds the chain id and contract address, so a signature for one deployment is not valid on another. Both parties sign the same structure hash, and that hash is the record id. Identifier fields are 32-byte strings; a longer identifier is stored as its hash, and the published file must carry the full text. Anyone can relay the transaction; the relayer has no authority. Records can be committed one at a time or in a batch that succeeds or fails as a whole.

One record is live per key at any time. A new record for a key whose window overlaps a live record is rejected, unless it names that record in its supersedes field; the old record then ends where the new one starts. Both parties sign the new record, so an amendment carries both signatures and the history of a key is a chain of records. A verifier therefore never finds two answers for one date.

The prototype network is Hyperledger Besu with IBFT 2.0 consensus [22] and four validators, which stand for CMS, a provider association, a payer association, and an auditor. Validators order transactions; they cannot sign a rate. The choice of a replicated ledger over a one-operator log is a governance choice, discussed below.

Verification

A verifier reads the registry once into an index by key, then checks each published line with one lookup. The verdict for a line is one of: match; amount differs; no record (the key was never signed); expired (signed, but no window covers the date); ambiguous (reserved; the registry now prevents it); or needs chargemaster (a percent rate whose list price is not available). For a fixed rate the expected amount is the signed value. For a percent-of-charges rate it is the signed percent of the list price for that code, setting, and unit in the chargemaster version the record names. A provider

line that publishes the percent is compared on the percent; a line that publishes only the dollar result is compared on the recomputed dollar.

When both files are verified, a pair of lines that disagree resolves to one of: both match the record (the files agree with the record and so with each other), payer file departs, provider file departs, or unsigned. Today such a pair resolves to nothing.

Evaluation design

Full-sheet run. We take the public file of UCHealth Yampa Valley Medical Center (Steamboat Springs, Colorado; file dated 2025-11-01; CMS format 3.0.0; type 2 NPI 1790787307). Every payer named in the file is registered with prototype keys. Every row with a percentage becomes a percent-of-charges record that names a signed chargemaster version; every other row with a dollar amount becomes a fixed record; rows with only an algorithm carry nothing a record can hold and are counted. One record is signed per key; a second row with the same key and a different value is a key collision and is counted. Both signatures come from prototype keys, so the run measures mechanism and cost, not whether the file is correct. We then verify every row of the file against the registry, and again after altering 1% of rows.

Pair run on real data. We take five providers' files (Table 2) and one Aetna Transparency in Coverage in-network file (plan cluster pl-1s7, dated 2026-10-05, 547 plans, 1.69 GB compressed). For each provider we keep the Aetna commercial rows on the 16 Trilliant codes and map the file's plan name to one label, "Aetna commercial"; the crosswalk is manual and listed in the code. From the payer file we keep prices for the five providers' NPIs on the same codes. We pair on the record key and report the Trilliant measure: exact match, within 10%, and more than 50% apart. This is the agreement before signing.

Synthetic attribution. For each paired key we sign one record with the provider's published amount as the agreed value. We then alter 10% of provider lines and 10% of payer lines at random by $\pm 10\%$, $+50\%$, or -50%, verify both copies, and report the attribution of each remaining difference. The signatures are prototype keys; the run shows the mechanism, not the parties' agreement.

Discovery. We record how many provider files could be found and parsed. The CMS locator file (`cms-hpt.txt`) at each provider's domain names the file.

All code, the contract, the parsers, and the run output are in the repository [23]. Tests cover commit and read, duplicate rejection, unauthorized and wrong-side keys, revocation and its time bound, delegation windows, root signing prohibition, domain binding, root replacement with delay and veto, overlapping windows, supersession, batch commit, and both verifier paths.

Results

Full-sheet run

The file has 633,324 rows. 165,513 rows carry only a chargemaster, revenue, or NDC code and no CPT, HCPCS, or MS-DRG code; a record needs a code both parties use, so these rows are outside the record format. The remaining 467,811 rows name 20 payers. 104,121 rows give an algorithm and no dollar or percent value. The rest reduce to 262,535 records: 201,394 fixed and 61,141 percent of charges. 14,432 rows have a key that already had a record with a different value. These are items with the same billing code, setting, class, and unit but a different list price and a different fixed amount; the key fields that the public formats offer do not separate them, and a record for them needs an item-level identifier.

Table 1 gives the cost. Signing ran at 1,852 signatures per second on one laptop. Commits ran 100 records per transaction with eight transactions in flight, at 61 records per second over the run, so the sheet takes 72.3 minutes. The rate fell as the ledger grew: 93 records per second over the first 20,000 records of the run, 61 over all of it. Each record costs 310,367 gas, 834 bytes of transaction input, and 17,945 bytes on one validator's disk, so the sheet adds 4,711 MB to each validator. Gas is reported as a measure of execution work; the network charges no fee.

Verification of the clean file returned 349,258 matches, 14,006 amount-differs verdicts, and 426 needs-chargemaster verdicts. The amount-differs verdicts are the key collisions: the file carries a second value for a key the registry holds once. The needs-chargemaster verdicts are collision rows that carry a dollar amount for a key whose record is a percent

Table 1: Full-sheet run: one provider file, 20 payers, prototype keys on both sides.

Rows in file	633,324
Rows with a CPT, HCPCS, or MS-DRG code	467,811
Rows with only an algorithm	104,121
Records signed (fixed / percent of charges)	262,535 (201,394 / 61,141)
Rows whose key collided with a different value	14,432
Signing, both sides	202 s (1,852 signatures/s)
Commit, 100 records per transaction	61 records/s (72.3 min for the sheet)
Gas per record	310,367
Transaction input per record	834 bytes (156 MB total)
Validator storage per record / for the sheet	17,945 bytes / 4,711 MB
Registry load into the verifier index	572 s
Verification of 363,690 rows	1.3 s (289,178 rows/s)
Clean copy: match / amount differs / other	349,258 / 14,006 / 426
Altered copy: rows altered / caught / already differed / not comparable	3,584 / 3,454 / 128 / 2

of charges; the verifier needs the list price to compute the expected amount. In the altered copy, 3,584 rows had their signed value changed; 3,454 changed from match to amount differs, 128 were collision rows that differed before the change, and 2 were needs-chargemaster rows. No altered row that matched before the change matched after it. This run is self-consistency: the records were minted from the rows they are checked against, so a match is expected. What it establishes is that the record format covers every row with a standard code and a value, that the key holds for all but 14,432 rows, and the cost of signing a full sheet.

Discovery

Of ten health systems tried on 2026-10-27, four served the CMS locator file. Of ten tried on 2026-11-03, four served a tall CSV (OHSU, Advocate Christ, UCHealth Memorial, Alta Bates Summit), four served JSON (NewYork-Presbyterian, Houston Methodist, Stanford, BJC), one served a wide CSV (UW Health), and one served an empty archive (Intermountain). The JSON files are valid under the rule and our parser does not read them. On the payer side, UnitedHealthcare publishes national files of 9 GB and more per plan, which we did not download; Aetna publishes 5,697 in-network files by plan cluster of about 1.7 GB each, with no state in the name. The pair run therefore covers one payer.

Pair run on real data

Table 2 gives, per provider, the keys on each side and the agreement on paired keys. The Aetna file lists 468 prices for the five providers' NPIs on the 16 codes, 232 after lines with the same key and value are merged. The crosswalk maps every Aetna commercial plan in the file to one label, so a key has 1.53 payer values on average; each provider line is paired with the nearest payer value of the same kind, dollars with dollars and percent of charges with percent of charges. Across providers, 52 provider lines paired. 33 carry the same kind of value on both sides; 19 carry a dollar on one side and a percent of charges on the other and cannot be compared. Of the comparable pairs, 45.5% matched exactly, 97.0% fell within 10%, and 0.0% differed by more than 50%. Paired against every payer value of the key instead of the nearest, 30.0% of 50 comparable pairs matched exactly. Table 3 gives the nearest-value measure by code group.

Table 2: Five providers against one Aetna file on the 16 Trilliant codes. Keys: distinct record keys on each side. Pairs: provider lines with a payer value for the same key; comparable: both sides give the same kind of value. Percentages are of comparable pairs.

Provider	File date	Codes	Provider keys	Payer keys	Pairs	Comparable	Exact %	Within 10%	Over 50%
UCHealth Yampa Valley	2025-11-01	0	0	44	0	0	–	–	–
UCHealth Memorial Central	2025-11-01	16	19	73	25	19	57.9	94.7	0.0
Advocate Christ	2026-11-03	8	8	16	8	8	37.5	100.0	0.0
OHSU	2026-03-23	16	46	24	16	5	0.0	100.0	0.0
Alta Bates Summit	2026-04-01	8	8	18	3	1	100.0	100.0	0.0

Table 3: Agreement by code group, all five providers.

Group	Pairs	Comparable	Exact %	Within 10%	Over 50%
Inpatient (8 MS-DRG)	27	17	29.4	100.0	0.0
GI (5 CPT)	19	10	70.0	90.0	0.0
Orthopedic (3 CPT)	6	6	50.0	100.0	0.0

Synthetic attribution

28 paired keys had a provider dollar amount; one record was signed for each. After alteration, 4 keys had a changed provider line, 3 a changed payer line, and 1 both. Verification attributed 3 differences to the payer file, 4 to the provider file, and 1 to both; 20 keys matched the record on both sides. Every altered line was attributed to the side that was altered. Before signing, the same pairs show 71.4% exact agreement and nothing about which side is wrong.

Key rotation

On the running network, one record was committed with the payer's signing key; then a new signing key was delegated and the old one revoked from a time ten seconds after the current block. After that time, a commit with the old key reverted with `SignerNotAuthorized` and the same commit with the new key succeeded. `authorizedAt` returned true for the old key at the commit time of its earlier record and false after the revocation, so the earlier record stays valid. A revocation dated before the current block reverted with `BackdatedRevocation`. A root replacement proposed by the registrar reverted on activation with `RootDelayNotPassed`; after the current root's veto, activation reverted with `NoPendingRoot` and the root was unchanged.

Discussion

What the run shows. The record format holds every row of a real provider file that carries a standard code and a value, which is 467,811 of 633,324 rows. The key fields of the public formats identify a rate for all but 14,432 rows. The rows they do not identify are items that share a billing code; the provider file knows them by chargemaster code, and the payer file does not know them at all. Any comparison of the two files is undefined for these rows, and the Trilliant measure counts them as disagreement. Signing and committing a full sheet is a batch job of 72.3 minutes on one laptop, and verifying a published file is a job of seconds once the index is loaded.

What the comparison shows. On the 16 codes, five providers' files and one Aetna file agree exactly on 45.5% of comparable pairs, and 19 of 52 pairs cannot be compared because one file gives a dollar amount and the other a percent of charges. The pairing itself required a manual crosswalk of plan names, under which one key has several payer values, and the payer file has no drug unit and no setting beyond the place-of-service code. These are the same gaps the record format closes: a record carries one plan id, one setting, one unit, and one date, agreed by both parties.

Why a replicated ledger. Kuo and colleagues' conditions apply: providers, payers, and CMS each produce their own record of the same term; the history must be append-only, because a rate that changes mid-year must keep its earlier record; and no party is accepted by the others as the operator of the record. A CMS-run signed log with a hash chain would give the same inclusion guarantee, and the registrar role in CORAL is a single point of trust for identity binding. The delay and veto on root replacement limit that trust to identity binding; the registrar cannot sign a rate, cannot remove a record, and cannot re-key an entity without a week's notice to that entity. Whether the regulator should also operate the log is a policy question; the design works under either answer.

Adoption. The second signature does not exist today, and the party that must add it is the one whose file is less likely to be checked. Three routes exist. CMS can require that a rate reported in either file cite a jointly signed record, as the CY2024 rule required an affirmation statement [11]; enforcement under 45 CFR 180.90 already covers file content. A payer can require the signed record as a contract term, because it gives the payer a defense against a provider file that overstates a rate. A provider association can run the registry for its members and invite payers, because the signed record settles which file is wrong before a dispute reaches a regulator. In each route the ledger changes nothing about how rates are negotiated; it changes what is published.

Limitations. Both signatures in every run come from prototype keys, so no run shows that a file is correct, only that a

file matches the record it was minted from. The chargemaster version signed is the SHA-256 of the provider file itself, which carries the list prices; a deployment would sign the chargemaster as a separate file. The record carries a contract version and dates that no public file holds; the runs set them to one label and one date. Percent-of-reference rates and algorithm-only rows (104,121 rows here) are not covered. The pair run covers one payer, five providers, and 16 codes, after a manual plan crosswalk, and the provider files are dated between 2025-11-01 and 2026-11-03 against a payer file dated 2026-10-05. The discovery rate shows that a national reproduction is a data engineering project before it is a study. The registry rejects overlapping windows but cannot detect a rate that both parties signed wrongly.

Conclusion

A negotiated rate is a term that two parties agreed, and the public record of it should carry both signatures. CORAL gives a record format that both public files map to, a registry with a key hierarchy that judges validity at commit time and keeps one live record per key, and a verifier that checks a published line with one lookup. On one real provider file the format covers every row with a standard code and a value, the key holds for all but 14,432 rows, and signing the sheet is a batch job of 72.3 minutes on one laptop. On five providers and one payer the files agree exactly on 45.5% of comparable pairs on the 16 codes that Trilliant used, and the signed record turns each remaining difference into a finding about one file.

References

[1] Centers for Medicare & Medicaid Services. Medicare and Medicaid Programs: CY 2020 Hospital Outpatient PPS Policy Changes and Payment Rates and Ambulatory Surgical Center Payment System Policy Changes and Payment Rates. Price Transparency Requirements for Hospitals To Make Standard Charges Public. Final rule. Fed Regist. 2019 Nov 27;84(229):65524–65606. Codified at 45 CFR Part 180.

[2] Internal Revenue Service, Employee Benefits Security Administration, Centers for Medicare & Medicaid Services. Transparency in Coverage. Final rule. Fed Regist. 2020 Nov 12;85(219):72158–72310.

[3] Patton K, Ikard M, Browne J, Oakes A. The majority of hospital and health plan reported rates diverge by more than 10% [Internet]. Trilliant Health; 2026 Apr 30 [cited 2026 Nov 3]. Available from: https://www.trillianthealth.com/market-research/studies/the-majority-of-hospital-and-health-plan-reported-rates-diverge-by-more-than-10

[4] Centers for Medicare & Medicaid Services. Hospital price transparency CSV data dictionary, version 3.0 [Internet]. [cited 2026 Nov 3]. Available from: https://github.com/CMSgov/hospital-price-transparency/tree/master/documentation/CSV

[5] Centers for Medicare & Medicaid Services. Transparency in coverage: in-network rate file schema [Internet]. [cited 2026 Nov 3]. Available from: https://github.com/CMSgov/price-transparency-guide/tree/master/schemas/in-network-rates

[6] Jiang JX, Polsky D, Littlejohn J, Wang Y, Zare H, Bai G. Factors associated with compliance to the hospital price transparency final rule: a national landscape study. J Gen Intern Med. 2022;37(14):3577–3584. doi:10.1007/s11606-021-07237-y

[7] Wang Y, Meiselbach M, Anderson GF, Bai G. Hospital pricing information consistent between Transparency-in-Coverage data and other commercial data sources. Health Aff Forefront. 2023 Nov 8. doi:10.1377/forefront.20231108.269718

[8] Henderson MA, Mouslim MC. Cross-validation of insurer and hospital price transparency data. Am J Manag Care. 2024;30(8):e247–e250. doi:10.37765/ajmc.2024.89594

[9] Turquoise Health. 2025 price transparency impact report [Internet]. 2025 Oct 29 [cited 2026 Nov 3]. Available from: https://turquoise.health/resources/reports/2025-price-transparency-impact-report

[10] PatientRightsAdvocate.org. Eighth hospital price transparency compliance report [Internet]. 2026 Sep [cited 2026 Nov 3]. Available from: https://www.patientrightsadvocate.org/eighth-annual-hospital-price-transparency-compliance-report-september-2026

[11] Centers for Medicare & Medicaid Services. Medicare Program: Hospital Outpatient Prospective Payment and Ambulatory Surgical Center Payment Systems; Quality Reporting Programs; Hospital Price Transparency. Final rule with comment period. Fed Regist. 2023 Nov 22;88(224):81540. CMS-1786-FC.

[12] Kuo TT, Kim HE, Ohno-Machado L. Blockchain distributed ledger technologies for biomedical and health care applications. J Am Med Inform Assoc. 2017;24(6):1211–1220. doi:10.1093/jamia/ocx068

[13] Change Healthcare. Change Healthcare announces general availability of first enterprise-scale blockchain solution for healthcare [press release on the Internet]. PR Newswire; 2018 Jan 8 [cited 2026 Nov 3]. Available from: https://www.prnewswire.com/news-releases/change-healthcare-announces-general-availability-of-first-enterprise-scale-blockchain-solution-for-healthcare-300578989.html

[14] Kalina IM. Another big blockchain initiative announced in health care insurance industry [Internet]. National Law Review; 2019 Feb 6 [cited 2026 Nov 3]. Available from: https://natlawreview.com/article/another-big-blockchain-initiative-announced-health-care-insurance-industry

[15] Jasper KA. 5 healthcare groups sign up for blockchain credentialing [Internet]. Ledger Insights; 2018 Nov 9 [cited 2026 Nov 3]. Available from: https://www.ledgerinsights.com/healthcare-blockchain-credentialing-provider/

[16] Laurie B. Certificate transparency. Commun ACM. 2014;57(10):40–46. doi:10.1145/2659897. Laurie B, Langley A, Kasper E. Certificate Transparency. RFC 6962. IETF; 2013 Jun.

[17] Birkholz H, Delignat-Lavaud A, Fournet C, Deshpande Y, Lasker S. An architecture for trustworthy and transparent digital supply chains. RFC 9943. IETF; 2026 Jun.

[18] Haeberlen A, Kouznetsov P, Druschel P. PeerReview: practical accountability for distributed systems. In: Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP); 2007 Oct; Stevenson, WA. p. 175–188. doi:10.1145/1294261.1294279

[19] Li J, Krohn M, Mazières D, Shasha D. Secure untrusted data repository (SUNDR). In: Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI); 2004 Dec; San Francisco. p. 121–136.

[20] Zieglmeier V. Appending data to blockchain is not sufficient for non-repudiation of receipt. arXiv:2308.04781 [cs.CR]. 2023 Aug 9.

[21] Bloemen R, Logvinov L, Evans J. EIP-712: Typed structured data hashing and signing [Internet]. Ethereum Improvement Proposals; 2017 Sep 12 [cited 2026 Nov 3]. Available from: https://eips.ethereum.org/EIPS/eip-712

[22] Hyperledger Besu. Configure IBFT 2.0 consensus [Internet]. [cited 2026 Nov 3]. Available from: https://besu.hyperledger.org/private-networks/how-to/configure/consensus/ibft

[23] Srivastava A. CORAL: Committed Ratesheet on Ledger [source code on the Internet]. 2026 [cited 2026 Nov 3]. Available from: https://github.com/AbhinavMir/coral

Working notes (remove before submission)

Everything built for this paper is recorded here. Dates are absolute. Numbers in the body come from `numbers.tex`, written by `scripts/make_numbers.py` from `results/*.json`. Repository: `https://github.com/AbhinavMir/coral`. Say provider, not hospital, except in a regulation name or a cited study.

State (2026-11-03)

- Paper body written: Introduction, Background, Related work, Methods, Results, Discussion, Conclusion, 23 references. Title unchanged; the review's suggestion (do not lead with "Private Blockchain") is not applied. Abhinav decides.
- Contract rewritten on 2026-11-03 after the review: revocation bound to block time, root replacement with a seven-day delay and a root veto, supersedes field, overlapping windows rejected, batch commit, batch read, drug unit and billing class in the key. 16 tests pass.
- Runs on 2026-11-03: full sheet (`results/evaluate_sheet.json`), five providers against one Aetna file on 16 codes plus the synthetic attribution run (`results/evaluate_pairs.json`), key lifecycle on the network (`results/key_events.json`). All numbers in the body come from these files.
- Not done: UnitedHealthcare payer side (files are 9 GB and more per plan); JSON provider files (four of ten systems); an on-ledger proposal record; witness checkpoints; a separate chargemaster file.

Problem

- Providers publish negotiated rates under 45 CFR 180. Payers publish in-network rates under Transparency in Coverage (85 FR 72158). Each file is produced by one party. No file carries the other party's signature.
- Trilliant Health, 30 April 2026: 450 hospitals, Aetna and UnitedHealthcare, 16 codes (8 MS-DRG: 189, 190, 193, 280, 291, 392, 552, 641; 5 GI CPT: 43239, 45380, 45385, 47562, 49650; 3 orthopedic CPT: 29827, 29881, 29916), data processed August 2025. Exact 9.0% inpatient, 10.8% GI, 11.7% orthopedic. Within 10%: 50.8%, 41.7%, 47.4%. Over 50% apart: 17.9% inpatient and GI. Ratio = hospital rate / plan rate; exact 1.0, near 0.9–1.1, material under 0.5 or over 1.5. The code list is in an image on the report page, not in its text.
- UCHealth Yampa Valley Medical Center file (2025-11-01, CMS 3.0.0, NPI 1790787307): 633,324 rows; every row billing class facility; settings inpatient 229,285 and outpatient 404,039; methodologies: percent of total billed charges 228,164, other (algorithm) 178,531, fee schedule 168,658, case rate 57,918, per diem 53; 30 payer/plan pairs; 165,513 rows have only CDM, RC, or NDC codes.

Terms

- Negotiated rate: the price a provider and a payer agreed for one code. Contractual. The CMS file calls it the payer-specific negotiated charge.
- Chargemaster: the provider's list price for every item. Set by the provider alone. The CMS file calls it the gross charge. A percent-of-charges rate is a percent of this number.
- Setting: inpatient or outpatient (CMS column; "both" expands to one line per setting). Billing class: facility or professional (CMS column; TiC `billing_class`). Drug unit: CMS `drug_unit_of_measurement` and `drug_type_of_measurement`, joined as "1 ML". TiC has no drug unit.
- Record key: payer, plan, provider TIN, NPI, code, modifier, setting, billing class, unit. Not in either file and not in the record: place of service beyond the setting split, revenue code, bundling, multiple-procedure reductions, outliers, carve-outs, per-diem tiers, escalators.
- A percent-of-charges row in the CMS file carries both the percent and the dollar result. The contract term is the percent; the record is percent of charges and the verifier recomputes the dollar from the row's gross charge. Signing the dollar instead produced 81,470 false key collisions on the first trial (2026-11-03); signing the percent leaves 14,432.

Prior art (searches 27 October and 3 November 2026)

- No paper or product found on jointly signed negotiated rates or a ledger that holds rate records.
- Claim and directory networks: Change Healthcare on Hyperledger Fabric (2018), Synaptic Health Alliance (provider directory), ProCredEx (credentials), MediLedger 340B (2021). None holds a rate with both signatures.
- File comparisons: Jiang et al. JGIM 2022 (compliance); Wang et al. Health Affairs Forefront 2023 (TiC consistent with commercial sources); Henderson and Mouslim AJMC 2024 (insurer vs hospital cross-validation); Turquoise 2025 impact report; PatientRightsAdvocate eighth report (September 2026, 49.4% of 2,000 hospitals compliant). No Jiang/Bai hospital-versus-insurer paper exists; the review's pointer was wrong.
- Transparency logs: Certificate Transparency, Trillian, SCITT (now RFC 9943, June 2026). PeerReview, SUNDR. Zieglmeier arXiv 2308.04781: an append is not proof of receipt.
- CY2024 OPPS rule (88 FR 81540): affirmation statement and estimated allowed amount; CY2026 replaced the estimated allowed amount with median, 10th, and 90th percentile allowed amounts from 1 January 2026.

Keys

- Identity roots: provider TIN (the Aetna file carries the EIN per provider group, so the payer side can name the same entity); payer id (prototype: opaque string; deployment: HIOS or X12 payer id). Registrar: CMS, binds entity id to root key once, never signs a rate.
- Root key offline, signs delegations, revocations, and root vetoes only. Signing keys delegated for a window. `authorizedAt(entity, key, t)` answers from the ledger.
- Validity judged at the block timestamp of the commit. `revoke` rejects `effectiveFrom` before `block.timestamp` (`BackdatedRevocation`), so a root cannot repudiate a record its key signed. The app's rotate action revokes from now plus ten seconds.
- Root replacement: `proposeRoot` (registrar), `activateRoot` after `ROOT_DELAY` = 7 days, `vetoRoot` by the current root (EIP-712 `RootVeto`). Old delegations stay until the new root revokes them.
- EIP-712 typed data for every record; domain bound to chain id and contract address. Both parties sign the same struct hash; the hash is the record id. First signature held off-ledger in `build/pending.json` until the second party countersigns.

Design

- Rate struct: 16 fields (9 key, 7 value including `supersedes`). `rateId` hashes two `abi.encode` halves joined with `bytes.concat`, which equals one encode of all fields; the contract compiles with `viaIR` to avoid stack-too-deep.
- One live record per key: `keyRecords[keyHash]` lists ids; `_checkOverlap` rejects a window that overlaps a live window (`OverlappingWindow`); `supersedes` names the record to end, which must share the key and not be superseded already; the old record's effective end becomes the new start (`RateSuperseded` event; `effectiveEnd(id)`).
- Identifiers are `bytes32`. A string over 32 bytes (the Yampa file has the plan "Narrow Network Exclusive Plan (EPO)", 35 bytes) is stored as its keccak hash; the ledger cannot return the text. `keyform` puts both sides of a comparison in the ledger's form (text, or hex of the hash). Found on 2026-11-03 when the resumed sheet run hit `AlreadyCommitted` on records the index had read back under a different key.
- The full-sheet run was interrupted twice (a transient "latest block not found" from Besu at 75,200 records; then the key-form defect) and resumed; the script skips keys already on the ledger, and `retry` covers null RPC answers during block import. Commit rate and per-record figures come from the third run (187,025 records); the minutes-for-the-sheet figure is records divided by that rate. The rate fell with ledger size: the first run on

an empty ledger ran between 165 and 190 records per second; the third run ran 93 over its first 20,000 records and 61 over all of it. The verify step was rerun alone (`-skip-commit`) after the tamper step was corrected to alter the field the key's record holds (the percent when the record is percent of charges, else the dollar). The first version altered the dollar on every row, which a percent record does not sign, and reported 1,321 of 3,584 altered rows as missed; a second version altered the percent whenever the row had one and missed 5 rows; those were drug amounts of 3 or 4 cents that a 10 percent change rounded back to the same cent, so the tamper step now moves a value by at least one cent or one basis point. `make_numbers.py` asserts that no altered row still matches.

- `commitRates` commits a batch atomically. Besu's default transaction size limit (128 KB) allows about 100 records per transaction at 834 bytes each; 250 failed with "Transaction size exceeds the maximum allowed size".
- `getRates(ids[])` returns bodies and commit data for many ids; the client reads 500 per call. Without it, loading 262k records took two RPC calls each.
- Verifier verdicts: match, amount differs, no record, expired, ambiguous (cannot occur now), needs chargemaster. Percent lines compared on the percent when the file gives it.

Build

- `contracts/RateRegistry.sol` (Solidity 0.8.24, viaIR, 11,901 bytes, 59 ABI entries). `coral/typed.py` (types, `Rate.key()`, `sign_root_veto`). `coral/chain.py` (`get_rates`, `commit_rates`, `propose/activate/v` `RateRecord.effective_end`, `covers`). `coral/verify.py` (`Line.key()`). `coral/mrf.py` (`Skipped` counts; `normalize_modifiers`; unit and billing class; payer parser: gzip, code filter, remote `location` references counted as `provider_reference_remote`, no service code counted as `setting_unknown`, place of service 21 inpatient and 19/22/23/24 outpatient, per diem accepted as an amount). `coral/match.py` keys on `Line.key()`.
- Network: Hyperledger Besu 26.10.2, IBFT 2.0, four validators in Docker, chain id 2741, zero base fee. `Chain.logs` scans from the deploy block in 4000-block ranges (Besu caps `eth_getLogs` at 5000 blocks).
- Scripts: `bootstrap.py` (deploy, register three entities), `seed_examples.py` (ten examples plus the chargemaster hash; J1650 carries unit "1 ML"), `evaluate_sheet.py` (full sheet; registers every payer in the file; signs with a process pool; batches of 100, eight in flight; verifies clean and 1% altered copies; disk delta of the CMS validator), `evaluate_pairs.py` (five providers vs Aetna on 16 codes; synthetic attribution), `make_numbers.py`. `evaluate_provider.py` (the 200-record run of 2026-10-27) was removed; its output stays in `results/evaluate_provider.json`.
- App: provider/payer page (Negotiated rates with one search box, Pending, Propose with billing class, unit, and "replaces record", Chargemaster, Keys) and CMS page (Registry, Verify a published file with unit and class columns and skipped counts). The verify endpoint loads the ledger once into a `RateIndex`.
- Tests (16): commit and read, duplicate, unauthorized key, wrong-side key, revocation, backdated revocation rejected, delegation window, root cannot sign, domain binding, root replacement waits and can be vetoed, root replacement after delay, overlapping window rejected, supersede amends mid-window, batch commit (one bad signature fails the batch), verify fixed, verify percent of charges.

Data

- Provider files (`data/raw/`, not committed): UCHealth Yampa Valley (141 MB); `others/`: OHSU (284 MB, 2026-03-23, NPI 1609824010), Advocate Christ (143 MB, 2026-11-03, NPIs 1548375082, 1386188068, 1922542604), UCHealth Memorial Hospital Central (267 MB, 2025-11-01, NPI 1144397134), Alta Bates Summit, Summit campus (15 MB, 2026-04-01, eight NPIs). All CMS 3.0.0 tall CSV.
- Discovery, 2026-11-03: tall CSV from OHSU, Advocate, UCHealth, Sutter; JSON from NYP, Houston Methodist, Stanford, BJC; wide CSV from UW Health; empty zip from Intermountain. 2026-10-27: four of ten served `cms-hpt.txt` (NYP, UCHealth, OHSU, Houston Methodist).

- Payer files (`data/raw/tic/`): UHC index `https://transparency-in-coverage.uhc.com/api/v1/uhc/blob` (83,731 blobs, 2,977 in-network; commercial files 9.0–9.6 GB each; not downloaded). Aetna index via `health1.aetna.com/h` to `latest_metadata.json` (5,697 in-network files, 2026-10-05, by plan cluster); downloaded `pl-1s7-tr25` (547 plans, 414 Aetna Choice POS II, 1.69 GB gz). The file has inline `provider_references` with NPIs and EIN TINs.
- Aetna commercial rows per provider file: Yampa (Aetna, Commercial); Memorial (Aetna Core/Meritain, Commercial; Aetna ASA, Commercial not used); Advocate (Aetna, Commercial); OHSU (AETNA HEALTHCARE, AETNA); Alta Bates (Aetna, HMO/PPO). Crosswalk label on both sides: "Aetna commercial".

Results are in the body (Tables 1–3) and in `results/`. Earlier run, 2026-10-27, superseded: 200 UHC Broad Networks records one per block, 258,923 gas each, 1.99 s each, 200/200 match, 20/20 tampered caught.

- Pair run: the Aetna file gives a percent of charges for many of the 16 codes where the provider file gives a dollar (OHSU 11 of 16 pairs; Memorial 6 of 25). These pairs are reported as not comparable; the percentages are of comparable pairs only. Each provider line is paired with the nearest payer value of the same kind because the crosswalk collapses every Aetna commercial plan to one label (1.53 payer values per paired key); the line-by-line measure against every payer value is reported beside it. Yampa Valley's Aetna commercial rows have none of the 16 codes (its Medicare Advantage rows do), so it contributes no pairs.
- Synthetic attribution: 28 paired keys had a provider dollar amount; 28 records signed; 4 provider-altered, 3 payer-altered, 1 both; attribution 20 both match, 3 payer departs, 4 provider departs, 1 both depart.
- Key events (`key_events.json`): backdated revoke reverted `BackdatedRevocation` (0x1532513f); commit with the revoked key reverted `SignerNotAuthorized` (0x3b94fe24); commit with the new key succeeded; `authorizedAt` true at the earlier commit time, false after; activation before the delay reverted `RootDelayNotPassed` (0xe90309b1); after the veto `NoPendingRoot` (0x2a01895e); root unchanged. The script rotates the PAYER:AETNA signing key in `build/keys.json`.

Review (2026-11-02, simulated PI) and what was done

- Abstract overclaims: rewritten to the runs. Done.
- Circular 200-record evaluation: replaced by the full-sheet run reported as self-consistency plus cost, with the key-collision and no-code findings. Done.
- `revoke` backdating: bound to block time. Done. `replaceRoot` without delay: delay and veto. Done. No correction path: supersedes and overlap rejection. Done.
- Drug unit and billing class in the key: done. Payer parser defects (remote references, missing service code): counted, not guessed. Done.
- Invented fields (contract version, start, end): stated in Methods and Limitations. Chargemaster is the MRF's own hash: stated in Limitations.
- Attribution as a synthetic run: done (`evaluate_pairs.py`, part 2).
- 450-hospital reproduction replaced by five providers, one payer, 16 codes, with discovery reported. Done. UHC side not done (file size).
- Related work added (Jiang 2022; Wang 2023; Henderson 2024; Turquoise; PRA; OPPS CY2024; Kuo 2017; ProCredEx; Zieglmeier). The Jiang/Bai comparison paper the review named does not exist.
- Writing: gas kept in Table 1 as a work measure with a note that the network charges no fee; EIP-712 named once in Methods; adoption paragraph added; "departs from the signed record" used. Title not changed.

Open items

- UHC payer side: one 9 GB file for the UHC plans in the Yampa file.
- JSON provider files: a parser for the CMS JSON schema would double the discovery rate.
- Item-level key: the 14,432 collision rows need a chargemaster item id in the key or a rule that one code has one rate.
- On-ledger proposal record (who refused to sign); witness checkpoints; a separate chargemaster file instead of the MRF hash.
- Cut to 10 pages and run a claim audit before submission.

Venue

- AMIA Annual Symposium 2027, Paper - Regular: 10 pages including references and figures, abstract 125–150 words, one column, Letter, Vancouver references. Symposium 6–10 November 2027, San Diego. Call for papers expected March 2027.
- Alternatives: IEEE ICBC 2027 (deadline 14 January 2027), Blockchain in Healthcare Today, Journal of Medical Systems.